

IMPLEMENTATION MILESTONE REPORT

Hostel Buddy

Smart Hostel Complaint Management System

30% · 60% · 80% · 100%

Group 19

Department of Computer Science and Engineering

IIT Kottayam

Reporting period: 21 July — 31 August 2026

1. Team Members

#	Member Name	Roll Number
1	Pranjal Tyagi	2024BCD0053
2	Dinesh Saini	2024BCD0033
3	Ajinkya	2024BCS0173
4	Saurabh	2024BCS0185
5	Vijay	2024BCS0249

2. Purpose of This Report

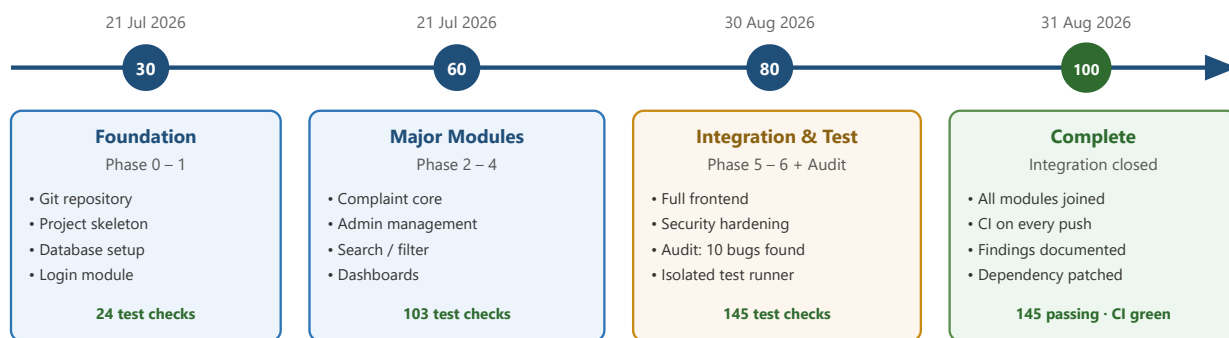
This report records how **Hostel Buddy** was built, milestone by milestone. For each of the four checkpoints — **30%, 60%, 80% and 100%** — it states which development phases were completed, how the architecture grew, which bugs were found and fixed, and how the work was verified. Every claim is traceable to a commit in the project's Git repository.

Development model used

The project followed the **Incremental Model**. Requirements were fixed first in an approved SRS, the architecture was designed before coding, and construction then proceeded in **seven increments (Phase 0 – Phase 6)**. Each increment produced **working, tested software** rather than a half-finished layer, so the system was demonstrable at every checkpoint.

Why incremental and not Waterfall? Because defects were caught *during* construction and fixed before the next phase began — not discovered at the end. The repository shows this directly: a dedicated `Review fixes` commit sits between Phase 2 and Phase 3.

3. Milestone Overview



Each milestone delivered running, tested software — never a partially built layer.

Figure 1 — The four implementation milestones and what each delivered.

Milestone	Phases	Date	Outcome	Status
30%	Phase 0 – 1	21 Jul 2026	Repository, skeleton, database and login working	Completed
60%	Phase 2 – 4	21 Jul 2026	All major functional modules working through the API	Completed
80%	Phase 5 – 6 + Audit	30 Aug 2026	Full UI integrated; audit performed; test suite rebuilt	Completed
100%	Integration close-out	31 Aug 2026	Everything integrated, automated in CI and documented	Completed

30% Foundation — Git repository · Project skeleton · Database · Login module

Phases 0 – 1 · completed 21 July 2026

4.1 What this milestone means in simple terms

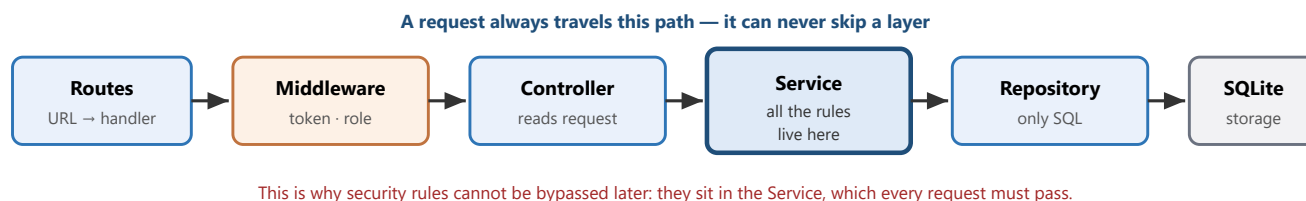
At 30% the project has **no features a student would notice yet** — but it has everything the features will stand on. The repository exists, the folder structure is decided, the database tables are created, and one complete module (login) works end to end. Choosing login first was deliberate: **every later feature needs to know who is asking**.

4.2 Phases completed

Phase	Commit	What was built
Phase 0 Foundation	8eef41d	Node/Express project, folder skeleton, environment configuration, database connection and <code>schema.sql</code> applied on startup, central error handler, request logger, health-check endpoint
Phase 1 Authentication	e4b145e	Users table access, registration with bcrypt hashing, login issuing a JWT, <code>requireAuth</code> / <code>requireRole</code> guards, seeded admin account, profile endpoints

4.3 How the architecture was set up here

The **layered architecture** was established at this milestone and never changed afterwards — later phases only added modules that follow the same shape:



This is why security rules cannot be bypassed later: they sit in the Service, which every request must pass.

Figure 2 — The layered request path fixed at the 30% milestone.

Database: two tables were created — `users` (in active use) and `complaints` (created and waiting for Phase 2). Constraints were placed in the schema itself: unique email, a foreign key from complaint to user, and `CHECK` rules limiting role, category and status to valid values.

4.4 Bugs and problems at this stage

B2 — Login revealed which emails were registered. Different messages for "no such user" and "wrong password" let an attacker discover valid accounts. **Fix:** one generic 401 Invalid email or password for both. **Lesson:** security errors should be deliberately vague; usability errors specific.

Environment problem — the database driver would not install. The design named `better-sqlite3`, which compiles C++ and failed without Visual Studio build tools. **Fix:** switched to Node's built-in `node:sqlite`. **Benefit:** any team member can clone and run the project with no compiler. Recorded as decision **D5**.

4.5 Verification

The login module was covered by `tests/auth.test.js` — **24 automated checks**, all passing, including that the password hash never appears in a response, that a wrong password gives a generic error, and that a student cannot reach admin-only endpoints.

Viva point. "We built authentication before any feature, because authorisation is not something you can add afterwards — every later endpoint needed a trustworthy answer to *who is calling*."

60% Major Modules Functional

Phases 2 – 4 · completed 21 July 2026

5.1 What this milestone means in simple terms

At 60% the system **does its actual job**. A student can raise a complaint and track it; an administrator can find any complaint, move it through its lifecycle and see statistics. Everything works through the API — the browser screens come later. In other words, the **product logic is complete and proven** even though it is not yet pretty.

5.2 Phases completed

Phase	Commit	What was built
Phase 2 Complaint core	342a397	Raise a complaint (auto Pending), list own complaints, view one, edit and delete only while Pending , optional image upload
<i>Review</i>	7668bc8	Mid-build review and fixes (see 5.4)
Phase 3 Admin management	b7f6bac	List all complaints joined with student name/room, search by ID/name/room, filter by category and status, pagination, status changes with remarks
Phase 4 Dashboards	38cc358	Student statistics and admin statistics — totals, counts per status, category distribution, recent complaints

5.3 How the architecture developed

The structure did not change — it **replicated**. Each new module repeated the same four-file pattern, which is exactly the benefit the layered design was chosen for.

Module added	Design decision worth explaining
Complaint Management	The rule "edit only while Pending" was placed in the Service , not the route. A route-level check protects one route; a service invariant holds for every caller, present or future.
Dashboard	Given no repository of its own . It reads counts published by the Complaint and User modules, so reporting cannot break when a table changes. Counting is done in SQL (<code>GROUP BY</code>), not in JavaScript.

Source files grew from **20 to 29** across these phases, while the number of architectural layers stayed at five — growth was horizontal (more modules), not vertical (more complexity).

5.4 Bugs found and fixed at this stage

A deliberate review was carried out after Phase 2, before continuing. This is the clearest evidence of incremental development in the repository — the fixes have their own commit, 7668bc8.

Problem found	Why it happened	Fix
Seed script was broken	<code>npm run seed</code> pointed at a file that did not exist — an advertised command that crashed	Wrote the seed script; it now loads an admin, 5 students and 13 complaints, and is safe to re-run
Insecure defaults could reach production	A missing JWT secret or default admin password only produced a warning	The server now refuses to start in production with insecure settings

Duplicate-email race	Two simultaneous registrations could slip past the check and hit the database constraint, returning a 500	The constraint violation is caught and returned as a clean 409
-----------------------------	---	--

B3 — resolved_at was overwritten on every later update. Re-saving a Resolved complaint kept moving its resolution time, and closing it reset the date. **Root cause:** the field was assigned unconditionally instead of "set once".

Fix: it is written only the first time a complaint reaches Resolved. **Lesson:** "the first time X happens" is a different rule from "whenever X is true".

B4 — The client was trusted with its own status. A crafted request could create a complaint already marked *Resolved*, skipping the workflow. **Fix:** the service **ignores** any status sent by the client and forces Pending. **Lesson:** never let the client set a field the workflow owns.

B6 — Tests silently depended on leftover data. Suites passed alone but failed when run after one another, because they assumed a pristine database. **Fix:** every suite now creates its own students and complaints and asserts *changes* rather than absolute totals. **Lesson:** a test that only passes first is not really passing.

5.5 Verification

The suite grew to **103 checks across 4 files** — authentication (24), complaints (24), admin (32) and dashboard (23) — and passed in any order.

Viva point. "At 60% we could demonstrate the complete complaint lifecycle through the API. We deliberately proved the logic before building screens, so that any bug we found afterwards was a *display* problem, not a *rules* problem."

80% Integration Testing & Test-Case Preparation

Phases 5 – 6 + Audit pass · completed 30 August 2026

6.1 What this milestone means in simple terms

At 80% the browser screens were joined to the working API, so the system became a **real product** instead of an API. Then — importantly — we stopped adding features and **attacked our own application**. This audit is where most of the serious bugs were found. The suites had been green the whole time, because they tested the paths we designed; the audit tested the paths we had *not*.

6.2 Phases completed

Work	Commit	What was done
Phase 5 Frontend UI	7c42abf	Nine pages built to the SRS wireframes — landing, login, register, student dashboard, raise complaint, my complaints, profile, admin dashboard (Chart.js), admin complaints
Phase 6 Hardening	4844a74	Security headers and Content-Security-Policy, rate limiting, real performance measurements, screenshots, documentation reconciled
Audit fixes	d070669 1573dce 0d22041	Type-based validation and lifecycle enforcement; upload, error-response, rate-limit and 404 hardening; admin students page and three frontend defects
Test rebuild	287ed4b	Isolated test runner with its own server, database and upload directory; new regression suite

6.3 The audit — how we tested

The running application was attacked deliberately: **237 probe requests** with malformed and hostile input, followed by a browser walkthrough of every page at desktop and mobile widths.

What held: SQL injection, cross-site scripting, JWT forgery, mass assignment, ownership isolation and path traversal — all resisted. **Zero 5xx responses** were produced.

What broke: the ten defects below. Each one is now pinned by a permanent check in `tests/regression.test.js` so it cannot return.

6.4 Bugs found by the audit

ID	Bug	Root cause	Fix
B8	A JSON object was stored as the text "[object Object]" — registering with an object as the name created a real account with that junk name	Every validator coerced first and checked second. <code>String({})</code> is non-empty, so it passed. The correct helper existed but was never called	Rewrote validation around explicit type checks; wrong type is now a <code>400</code> , never coerced
B9	Any status could jump to any other — a Closed complaint could go back to Pending, producing a complaint that was "Pending with a resolution date"	Only the status <i>vocabulary</i> was enforced, not the <i>ordering</i> . Validating set membership felt like validating the rule	Added a transition guard: position may stay or increase, never decrease; backwards returns <code>409</code>
B10	Tests polluted the demo database and needed a		

	manually started server; one test opened the database file directly	Tests were written as "point them at whatever is on port 4000" — convenient once, coupled forever	New runner starts its own server on a throwaway database, then deletes it
B11	A requirement was invisible — the admin student list (FR-17) worked on the server but no page ever called it	Built during the API phase; the UI phase never circled back. Nothing failed because nothing tested it end to end	Added the admin students page and navigation link
B12	Upload checking trusted the browser — a text file declared as an image was accepted and stored	The declared MIME type was treated as a fact about the file rather than a claim by the client	The file's first bytes are read and the real format identified; impostors are deleted
B13	Framework internals leaked — a malformed request returned the JSON parser's own error text	The handler returned the raw message for anything under 500 — right for our errors, wrong for the framework's	Only our own errors keep their message; everything else is mapped to written text
B14	Search wildcards — typing % in the admin search returned every complaint	The term went to SQL LIKE . Parameterisation stops injection but not LIKE 's own metacharacters	Wildcards are escaped and every LIKE carries an explicit ESCAPE clause
B15	Long passwords were silently truncated — an 80-character password could be entered with only its first 72	bcrypt hashes at most 72 bytes and ignores the rest; we set a minimum length but no maximum	Passwords over 72 bytes are rejected with a clear message
B16	Brute-force protection never actually ran — the rate limiter was enabled only in production, and the app is demonstrated in development	"Don't throttle development" was reasonable; implementing it as "production only" quietly meant "never, in practice"	Active in development and production; disabled only for the test runner. Counts <i>failed</i> attempts only
B17	Six smaller defects — page numbers past the end echoed back, an image could be attached but never removed, unknown pages showed Express's stock error, the admin dialog's timer could close the wrong complaint, action buttons sat off-screen on a phone, and duplicated code	Each self-contained; none touched business logic	All fixed and covered by regression checks

Viva point — the most important lesson of the project. "Every one of these was found by *running* the application against hostile input, not by reading the code. Our test suite was green the entire time, because it tested the paths we designed. That is exactly the input an audit must not limit itself to."

6.5 Test cases prepared

The test infrastructure was rebuilt so the suite is **self-contained and repeatable**:

- `tests/run.js` starts the application on port **4010** with a throwaway database (`data/test.db`) and upload directory, waits for the health check, runs every suite, then shuts down and deletes both.
- A fifth suite, `tests/regression.test.js` (**41 checks**), pins every audit finding so a fixed bug cannot silently return.
- One command — `npm test` — now runs everything with no manual setup.

Suite	Checks	What it verifies
<code>auth.test.js</code>	24	Registration, login, token handling, role guards
<code>complaints.test.js</code>	25	Raise, view, edit/delete Pending-only, ownership, image upload
<code>admin.test.js</code>	32	List all, search, filter, pagination, status lifecycle, remarks
<code>dashboard.test.js</code>	23	Student and admin statistics, category distribution, guards

regression.test.js	41	One or more checks for every audit finding B8 – B17
Total	145	All passing

100% Complete System Integration

Integration close-out · completed 31 August 2026

7.1 What this milestone means in simple terms

At 100% every module is joined into one working system, the whole suite runs **automatically on every push**, and the engineering record is complete. Nothing is left half-connected: each requirement is reachable from the interface, verified by a test, and documented.

7.2 Work completed

Commit	Work	Why it mattered
8231f58	Audit findings documented	All ten defects recorded with root cause, fix and lesson — the engineering log a reviewer can inspect
b2dd98c	Continuous Integration on GitHub Actions	The suite now runs on every push and pull request . Because the runner is self-contained, CI needs nothing but Node — no services, no fixtures
21f86d3	Screenshots regenerated + capture tool	Documentation matches the shipped product, including the new pages; screenshots can be regenerated on demand
624a517	Corrected a claim in the performance review	A document stated something no longer true about how Chart.js is delivered — accuracy of the record
712bd9d	Dependency security patch	Updated a package to clear a high-severity advisory — supply-chain hygiene, not just our own code

7.3 Integration proof

Integration point	Verified by
Frontend ↔ API	All nine pages call the API through a single wrapper that attaches the token and handles session expiry; walkthrough at desktop and mobile widths
API ↔ Database	145 automated checks exercising every endpoint against a real database
Student ↔ Administrator	End-to-end lifecycle test: a student raises a complaint, the admin advances it, the student sees the new status, remarks and resolution date
Requirement ↔ Interface	Audit finding B11 closed the last gap — every requirement is now reachable from a screen, not only from a route
Repository ↔ CI	GitHub Actions runs the full suite on every push; a red build blocks the change

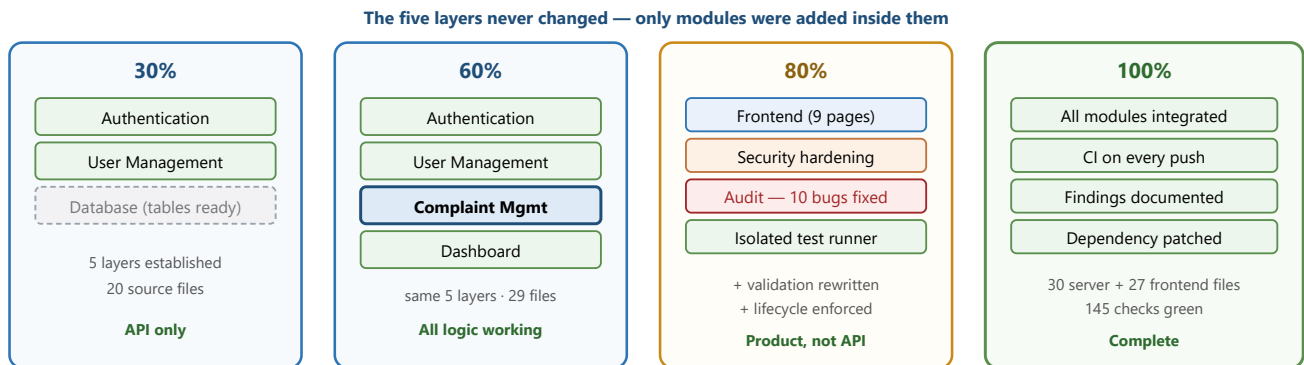
7.4 Final state of the system

Measure	Value
Development phases completed	7 (Phase 0 – Phase 6) + audit and close-out
Functional requirements implemented	41 of 41
Automated checks	145, all passing, in 5 suites
Bugs found and fixed (documented)	17 defects + 5 design decisions recorded
Application source files	30 server-side, 27 frontend
Continuous integration	Active on every push and pull request

Known unfixed defects	None
-----------------------	------

8. How the Architecture Developed

The single most important architectural fact is that **the layered structure was decided once, at 30%, and never had to be changed**. Later milestones added modules into that structure rather than reshaping it.



Growth was horizontal (more modules), not vertical (more complexity) — which is what made the audit fixes cheap.
Every audit fix touched one layer only: validation in Services, uploads in Middleware, wildcards in the Repository.

Figure 3 — Architecture growth across the four milestones.

9. All Defects at a Glance

Seventeen defects were found, fixed and documented across the project. Grouping them shows *when* each kind of problem surfaces.

Milestone	Defects	Character of the problems found
30%	B2 + environment	Security disclosure and tooling. Login revealed valid emails; the database driver would not install without a compiler
60%	B1, B3, B4, B5, B6 + 3 review fixes	Business-rule correctness. Timestamps overwritten, client-supplied status trusted, broken seed script, insecure production defaults, order-dependent tests
80%	B7, B8 – B17	Everything the happy path never touches. Type confusion, unenforced state ordering, spoofed uploads, leaked framework errors, search wildcards, truncated passwords, disabled rate limiting, an unreachable requirement
100%	Dependency advisory	Supply chain. A third-party package with a known high-severity vulnerability

The pattern worth explaining in the viva. Early milestones produced *logic* bugs, which our own tests caught. The 80% audit produced *assumption* bugs — places where we had trusted the client, trusted a type, or trusted that a documented rule was an enforced rule. Tests written from a design cannot find those, because they share the design's assumptions. Only adversarial testing does.

10. Quick Answers for the Viva

Question	Answer
Which SDLC model did you use?	The Incremental Model . Requirements were baselined in an approved SRS, the architecture was designed up front, and construction proceeded in seven increments — each delivering working, tested software, with review checkpoints between them.
Why is login the 30% milestone?	Because authorisation cannot be added afterwards. Every later endpoint needs a trustworthy answer to "who is calling", so authentication had to exist before any feature could be secured.
What does "major modules functional" mean at 60%?	The complete complaint lifecycle worked through the API — raise, track, search, advance status, view statistics — verified by 103 automated checks, before any screen was built.
How did you test the system?	Three ways: 145 automated integration checks across 5 suites; a deliberate audit of 237 hostile probe requests plus a full browser walkthrough; and continuous integration that runs everything on every push.
What was your most serious bug?	B9 — status transitions were unordered, so a Closed complaint could go back to Pending and end up "Pending with a resolution date". The lifecycle was drawn in the diagram but only the vocabulary was enforced in code, not the ordering.
Which bug taught you the most?	B16 — our rate limiting only ran in production, and we never run production. We had designed, documented and presented a protection that was never actually active. Security that only exists in an environment you never run is not security.
How did the architecture change over time?	It did not. The five layers were fixed at 30% and every later module was added inside them. That is why the audit fixes were cheap — each one touched a single layer.
How do you know it is really finished?	All 41 functional requirements are implemented, reachable from the interface, and covered by a passing test. CI is green, there are no known unfixed defects, and every defect we did find is documented with its root cause and fix.

11. Conclusion

Hostel Buddy was completed through four verified milestones over roughly six weeks. The foundation and authentication were established first (30%), the complete business logic was proven through the API next (60%), the interface was integrated and the system was then audited against hostile input (80%), and finally everything was joined, automated and documented (100%).

The project's strongest evidence is not that it works, but that **we know why it works**: seventeen defects are recorded with root cause and fix, one hundred and forty-five automated checks pin the behaviour, and every one of the audit findings has a permanent regression test standing guard over it.

Closing statement for the viva. "We treated finding bugs in our own work as a deliverable, not an embarrassment. The audit at 80% found ten defects that our green test suite had missed — and documenting each one with its root cause is what turned this from a project that runs into a project we understand."